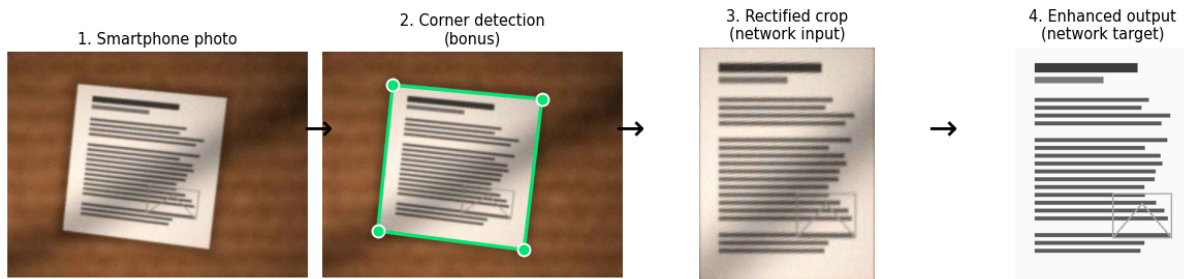




From raw photo to clean scan — the full pipeline



# Convolutional Neural Network Applications: Document Scanning & Enhancement

*Directed by Arshia Akbari Alagha*

Everyone has photographed a document with their phone: the page comes out tilted, dim, shadowed, and barely readable. Applications such as CamScanner solve this by locating the page, rectifying it, and enhancing it into a crisp, scanner-like image. In this project, you will build the heart of such an application yourself.

Image-to-image translation is a fundamental computer vision task in which a network maps an input image to a corresponding output image, pixel by pixel. In document enhancement, the goal is to transform a *degraded photograph* of a page into its *clean, scan-quality* counterpart, producing a restored image where the text is sharp and the lighting is uniform. This project has two mandatory tasks: the **enhancement network** and the **four-corner detection network**. The bonus part is joining them into a fully automatic, *end-to-end scanner*: raw photo in, clean scan out.



# 1. Preparing Dataset

Unlike a typical project, you will not photograph thousands of documents. You will be given clean scans, and you will learn the single most valuable trick in data-centric deep learning: **generating your own supervised dataset synthetically.**

## 1.1. The provided dataset

The teaching staff will provide you with:

- A collection of **clean, flat, high-resolution document scans**. These are your ground truth targets for training.

The real-world test set, however, is yours to create:

- **Task:** Collect 10–15 real smartphone photos of your own documents — pages your model has never seen in any form: notes, book pages, letters, printed homework. These photos are reserved for testing only and measure the question that matters most: *does your model generalize to entirely new documents, not just new photos of familiar ones?* For each photo, also produce a reference scan of the same document using a scanning app (CamScanner, Adobe Scan, or your phone's built-in document scanner). This is not ground truth — it is a commercial baseline to compare yourself against. Reference scans are for evaluation only and must never be used for training.

 **Note:** Your model's final grade will be determined at the presentation, where the teaching staff will run your pipeline on new, unseen realistic photos. Your own test photos are your rehearsal for that moment — the closer they resemble reality, the fewer surprises on presentation day.

You need to download the dataset from the given platform before starting the next steps.

 **Hint:** Make your 10–15 photos genuinely diverse — this is your only preview of reality. Vary the lighting (daylight, warm indoor lamp, harsh overhead light, a shadow falling across the page), the viewpoint (angles, distances, slight rotations), the background (desk, carpet, cluttered table), and the camera behavior (slight shake, imperfect focus). Vary the documents too: dense text, sparse text, a figure, a colored logo. Then look closely at what you captured: whatever degradations you see — shadows, blur, color casts, perspective distortion — are exactly what your synthetic pipeline in Section 4 must reproduce. Produce the reference scan at the same time you take each photo, while the document is still in front of you — reconstructing them later is tedious.



## 1.2. Label the corners of the real photos

The real test photos come without annotations. It's time to label the four page corners using RoboFlow. RoboFlow is a powerful tool for annotating images, and its **keypoint annotation** mode is exactly what we need: for every photo, place four keypoints on the page corners in a **consistent order** (top-left, top-right, bottom-right, bottom-left). You can ask the TAs on how to label your dataset for detailed guidance. Similar tools such as CVAT or Label Studio are also acceptable.

 **Hint:** *Consistent corner ordering is not optional — if your labels mix up top-left and bottom-right across images, both the evaluation and the rectification in the bonus part will silently break.*

## 1.3. Generate training pairs synthetically

Here is the key insight of this project: **you never need to annotate the training set at all.** For each training sample, you will:

1. Take a clean scan and a random background image (a desk, a table, a carpet).
2. Choose **four random target points** in the background image. These four points define a perspective transform (a homography).
3. Warp the scan onto the background using `cv2.getPerspectiveTransform` and `cv2.warpPerspective`.
4. Degrade the result (Section 4 describes the degradation pipeline in detail).

Notice what you get for free: the four points *you chose in step 2* **are** the corner labels — pixel-perfect, with zero annotation effort. And because you know the homography, you can warp the degraded photo back to obtain a perfectly aligned (degraded input, clean target) pair for the enhancement network. The label generator and the data generator are the same function.

 **Implementation:** The enhancement network operates on the **rectified crop**, not on the raw photo. Its input is the degraded document warped back to a flat rectangle; its target is the original clean scan. This decouples the two mandatory tasks: the enhancement network and the corner detector are trained and evaluated independently, and are only chained together in the bonus part.



## 2. Preprocessing

After building your synthetic generator and labeling the real photos, the preprocessing phase prepares your data for the training phase. This step ensures your data is consistent, normalized, and ready for your chosen machine learning framework.

### 2.1. Share Your Project Link

Before diving into preprocessing, you need to share your work with the teaching assistants.

- **Task:** Upload the link of your Roboflow project (where you labeled the corner keypoints) to the designated location in Google Sheets.
- This allows the teaching assistants to review your corner annotations on the real test photos.

 **Hint:** Double-check that your Roboflow project is public or accessible to the TAs before submitting the link.

### 2.2. Data Preprocessing

Now that your generator and labels are ready, it's time to normalize your data and prepare it for training.

- **Objective:** Convert your images into a consistent format suitable for deep learning models.
- **Tools:** If using PyTorch, leverage `DataLoader` and `Dataset` classes to manage and preprocess your data. If using Keras/TensorFlow, use `tf.data.Dataset`, which offers similar functionality.
- **Steps:**
  1. **Parse the annotation export:** Export your Roboflow keypoint annotations (COCO keypoint JSON is recommended) and parse them with the `pycocotools` library or plain `json`. Store the four corners of every real photo as an ordered array of shape `(4, 2)`.
  2. **Wrap the synthetic generator in a Dataset:** Your `Dataset` class should generate samples **on the fly**: each `__getitem__` call composites a fresh (degraded input, clean target, corner coordinates) triple. This gives you a practically infinite training set without ever writing images to disk.
  3. **Resizing:** Standardize all inputs and targets to your model's required input size (e.g., 256×256 or 512×512 pixels).



**Important:** When resizing an image, its corner coordinates must be **scaled by the same factors**. A corner label that is not transformed together with its image is a wrong label.

4. **Normalization:** Scale your image's pixel values (e.g., divide by 255.0 to get values in the range  $[0, 1]$ ). **Important:** Normalize the corner coordinates as well (divide by image width and height) so that they live in  $[0, 1]$  — this makes the corner detection task resolution-independent.
5. **Build a Data Loader Pipeline:** For PyTorch, apply `transforms.Normalize(mean, std)` after resizing. For Keras/TensorFlow, use `tf.data` operations to normalize your data.

 **Hint:** Check the documentation for your chosen framework to confirm the expected channel ordering (CHW vs HWC) before you spend an afternoon debugging a transposed tensor.

## 2.3. Dataset Preparation

With your pipeline in place, organize your data. Each bucket is prepared differently.

- Split your synthetic data into training, validation, and test sets, and prepare your real photos for evaluation.
- **Guidelines:**
  - Split by source scan, not by generated sample — two degraded versions of the same page must never end up on different sides of a split. A reasonable division of the scan collection is 80% / 10% / 10%. The validation set is for monitoring and model selection during training; the test set is held out and touched once, at the end, to report final numbers.
  - Freeze the validation and test sets. Because your Dataset generates samples on the fly, a naive implementation produces different degradations every epoch — your validation curve would then measure the dice as much as the model. Generate the validation and test samples once with a fixed random seed (or write them to disk), so that every epoch, and every model you compare, is scored on identical images.
  - Your real photos form a fourth, separate evaluation set, used in its entirety. Never train on them — and never run the degradation pipeline on them: they arrive degraded by reality.



**Prepare them as follows:** for the enhancement network, rectify each photo using the corners you annotated in Section 1.2, then resize and normalize exactly as you do for synthetic inputs, and resize the reference scan to the same size so the two are directly comparable. For the corner detector, the input is the raw photo, resized and normalized, with its annotated corners scaled by the same factors (Section 2.2).

- Both mandatory tasks share this split: the same source scans, the same held-out photos.

 **Note:** A second, hidden test set — new realistic photos provided by the teaching staff at the presentation — will be used for grading.

## 2.4. Final Checks

Before moving to training, verify your preprocessing steps.

- Test your `DataLoader` (PyTorch) or `tf.data.Dataset` (Keras/TensorFlow) to ensure the dataset loads without errors.
- Visualize a few (input, target) pairs side by side and overlay the corner labels on the composited photos to check that everything is aligned.

### 3. Task 1: The Enhancement Network – Model implementation and training

With a well-prepared data pipeline, the next stage is to design, train, and evaluate the enhancement model — the first of the two main tasks. This section provides a detailed description of the model architecture, training procedure, and performance testing methodology.

#### 3.1. Design the Model Architecture

You will design your own image-to-image model from scratch. The general principle is to create an **encoder-decoder structure**: the encoder progressively downsamples the degraded input to capture context (where is the shadow? how strong is the blur?), and the decoder progressively upsamples back to a full-resolution clean image. You can use standard layers like Conv2D, MaxPooling2D, UpSampling2D (or transposed convolutions), and activation functions like ReLU to build your network. You will definitely want **skip connections** passing fine-grained information from the encoder to the decoder — text strokes are thin, and without skip connections they will not survive the bottleneck.

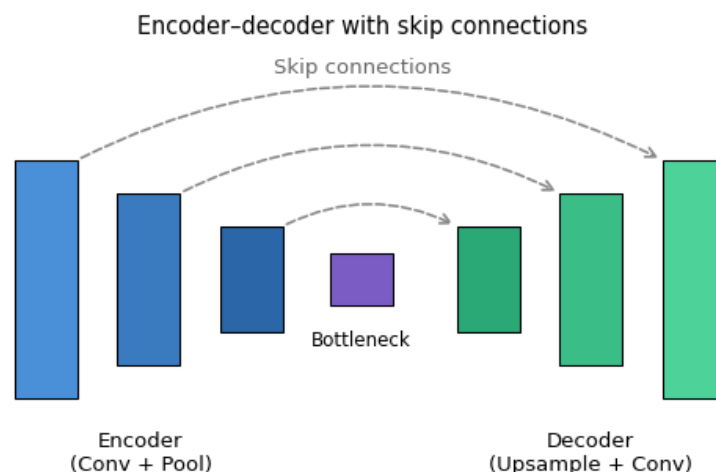


Figure 1 - U-Net architecture

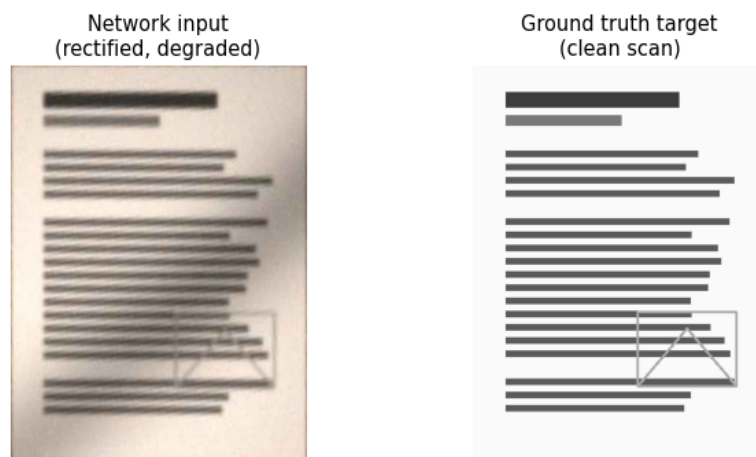


Figure 2 - Network criteria

 **Implementation:** We will not use pre-designed architectures (like importing a ready-made U-Net) or pre-trained weights. Furthermore, in this section, you should not use any dropout layers or other explicit regularization techniques. The emphasis is on good model training on the dataset and the original architecture design.

This entire architecture will be implemented using PyTorch or TensorFlow Keras layers in the `model.py` file.

### 3.2. Train the Model

The training process involves feeding our dataset to the model and optimizing its weights using a loss function. The implementation will be contained in `train.py`.

 **Implementation:** Before training begins, our dataset is split into a **training set** and a **validation set**. The model learns directly from the training set, while the validation set is used to evaluate its performance on unseen data at the end of each epoch. This process is crucial for monitoring the model's ability to generalize.

To monitor the learning process, we will plot the loss on both the training and validation sets against the number of epochs (your synthetic test set stays untouched until Section 3.3). Analyzing this graph is essential for diagnosing the model's behavior. Results and plots are required for next steps.

 **Option:** Because your Dataset generates samples on the fly, you can control the effective dataset size per epoch. Experiment with this: does the model benefit more from seeing many different degradations of few scans, or few degradations of many scans?

The key components of our training loop are:

- **Loss Function**
- **Optimizer**
- **Hyperparameters**

A standard pixel-wise loss like Mean Squared Error is known to produce **blurry outputs** in image restoration — and blur is precisely the enemy when the goal is readable text. Do you have any idea?

 **Hint:** Investigate the L1 loss, the (MS-)SSIM loss, and losses computed on image *gradients* (e.g., L1 between Sobel edge maps). Combinations of these are a well-known recipe in the image restoration literature. Text legibility lives in the edges.

 **Hint:** It is highly recommended to use cloud-based environments such as Google Colab for training. These platforms provide free access to GPUs, which can significantly speed up the training process compared to a standard CPU.

### 3.3. Evaluate the Model

Now that your model is trained, it's time to assess its performance. In this part, you will implement the evaluation step in the `evaluate.py` file by focusing on two widely-used metrics in image restoration: Peak Signal-to-Noise Ratio (PSNR) and the Structural Similarity Index (SSIM).

$$\text{PSNR} = 10 \log_{10} \left( \frac{\text{MAX}^2}{\text{MSE}} \right)$$

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}$$

Report PSNR and SSIM on all three synthetic buckets in a single table:

| Split      | PSNR | SSIM |
|------------|------|------|
| Training   |      |      |
| Validation |      |      |
| Test       |      |      |

Each row answers a different question. Training tells you how well the model fit what it was shown — a low score here is an optimization or capacity problem, not a generalization one. Validation is the number you steered on while training, so it is optimistic by construction. Test is the honest headline: source scans the model has never seen in any form. A large



training-vs-test gap means overfitting; a small gap with poor numbers everywhere means underfitting. The first row is your "do nothing" baseline — the metrics of the degraded input itself, before any enhancement, measured on the test bucket. Compute it first. If your model's scores are not clearly above this line, it is not earning its parameters.

On your real photos, no clean target exists — your documents were never scanned — so the table above cannot be extended to them. Evaluate against the commercial baseline instead:

1. **Qualitative:** rectify each photo with your annotated corners, run your model, and present (input, your output, reference scan) triplets. Where does your model match the app? Where does it fall short — and where, if anywhere, does it do better?
2. **Readability:** run an OCR engine on all three images — the rectified input, your enhanced output, and the reference scan — and compare the results, either as character error rate against text you transcribe for a few documents or as the engine's own confidence scores. Two questions matter: did your enhancement make the document more readable than the raw photo, and how close did you get to the commercial app?

Be fair to yourself when interpreting this: the reference has its own style — aggressive contrast, whitened background, sharpening — that differs from the flat scans you trained on, so 'different from CamScanner' is not the same as 'worse than CamScanner.' Judge by readability and by what you can see in the triplets. Finally, discuss the relationship between the table and the real photos: a model can top the synthetic test set and still fail on real photos — that gap is the central challenge of this project.

### 3.4. Pipeline the process

You should create a pipeline to automate the inference process for the enhancement network. This function will take a rectified document image as input and perform the following steps:

1. **Preprocess the image**
2. **Predict the enhanced image:** Pass the preprocessed image through the trained model to obtain the restored output.
3. **Post-process the output:** Resize the enhanced image back to the original dimensions and convert it to a standard 8-bit image for saving and visualization.
4. **Visualize the model's output**



## 4. Enhance the Dataset with Realistic Degradations

In this project, augmentation and dataset generation are one and the same: the richness of your degradation pipeline directly determines what your model can fix. A model that has never seen a shadow during training will not remove one at test time. In real phone photos, documents appear at various angles, distances, lighting conditions, and backgrounds.

### 4.1. Choosing Transformations

For document enhancement, we have selected the following degradations based on their relevance to the task:

- **Perspective warp:** Photos are taken at an angle, never perfectly overhead. Randomize the four target corners of the homography within sensible bounds. This simultaneously creates your corner labels (Section 1.3).
- **Scaling / resolution loss:** Documents can be photographed from far away. Downscale the image by a random factor (e.g., 2× to 4×) and upscale it back, simulating distance and limited sensor resolution.
- **Brightness, contrast, and color cast:** Lighting conditions differ due to light sources and time of day. Apply random brightness and contrast adjustments, and a random warm/cool color cast (scale the R and B channels by factors near 1).
- **Illumination gradients and shadows:** The most characteristic defect of real document photos. Multiply the image by a smooth random gradient, and composite soft random shadow shapes (e.g., blurred polygons) at reduced intensity.
- **Blur and noise:** Camera shake and sensor noise are unavoidable. Apply Gaussian or slight motion blur, then add Gaussian noise with a small standard deviation.
- **JPEG compression:** Phone images are stored compressed. Re-encode the image at a random JPEG quality (e.g., 30–80) using `cv2.imencode` and decode it back. We will avoid flipping: mirrored text is not something a document scanner should ever learn to “restore.”

### 4.2. Applying Transformations

When applying transformations, it is essential to maintain the alignment between inputs and targets. The geometric part (the perspective warp) must be **inverted exactly** — using the known homography — before the degraded image is paired with the clean target, while the photometric degradations (shadows, blur, noise, compression) are applied to the input **only** and never to the target. If the input and target drift out of alignment by even a few pixels, pixel-wise losses will punish the model for errors it did not make.



### 4.3. Degradation Pipeline

The generation pipeline for one training sample, applied in sequence:

1. Random perspective warp of the clean scan onto a random background (record the four corners).
2. Random downscale-upscale by a factor between 2 and 4.
3. Random brightness, contrast, and color-cast adjustment.
4. Multiplication by a random illumination gradient and compositing of soft shadows.
5. Gaussian blur followed by Gaussian noise.
6. JPEG re-encoding at a random quality between 30 and 80.

Each call to the generator produces a fresh sample, so your effective dataset size is limited only by training time.

### 4.4. Verification

To verify that the pipeline is correct, visually inspect a batch of generated samples: the degraded input, the clean target, and the corners overlaid on the composited photo. Ensure that warping the degraded photo back with the recorded homography aligns pixel-perfectly with the target. Additionally, place a few generated samples next to the real test photos — if a stranger can instantly tell which is which, your degradations are not yet realistic enough. Be cautious of excessive degradation, which might destroy the text entirely and leave the model nothing to recover.

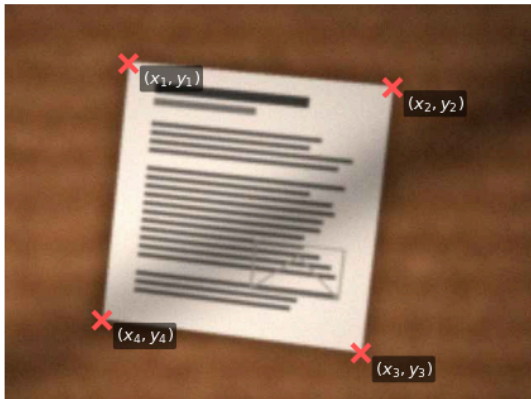
 **Hint:** Randomize *every* parameter within a range rather than fixing it. A model trained on one shadow direction learns that shadow direction, not shadows.

 **Implementation note:** You are not allowed to use any third-party libraries to handle transformations. You **MUST** use the techniques learned in the course and *OpenCV functions* to build the degradation pipeline.

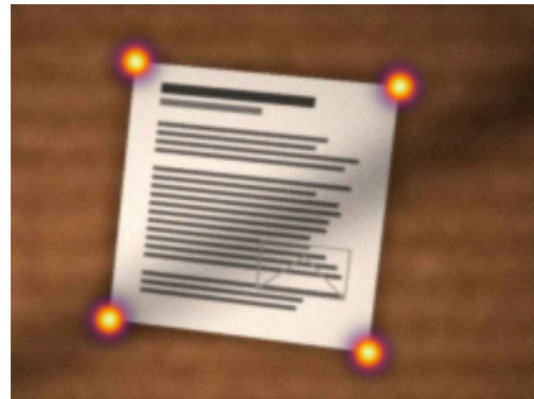
## 5. Task 2: Corner Detection – Two Roads to the Four Corners

So far, the rectification step used ground-truth corners. In this second mandatory task, you will predict the four page corners from the raw photo. There are two natural formulations of this problem, and — this is the interesting part — **you will implement both and let the experiments decide which one wins.**

Approach A — direct regression  
CNN head outputs 8 numbers



Approach B — heatmap regression  
four Gaussian heatmaps + soft-argmax



- **Approach A — Direct coordinate regression:** A CNN encoder followed by fully connected layers that output 8 numbers: the normalized (x, y) coordinates of the four corners. Train it with an L1 or L2 loss on the coordinates. Simple to implement — but is it easy to train well?
- **Approach B — Heatmap regression:** Reuse your encoder-decoder machinery from Section 3 to predict **four heatmaps**, one per corner, each containing a Gaussian blob centered on the true corner location. At inference, extract the coordinates with an argmax (or a *soft-argmax* if you want the extraction to be differentiable). Train it with a pixel-wise loss on the heatmaps.

 **Implementation:** As in Section 3, do not use pre-trained weights or dropout layers here — first versions of both corner detectors are built clean. Regularization comes in Section 6.

Compare the two approaches on the real, Roboflow-labeled test photos, and on the synthetic test set, where labels are exact, using the **mean corner localization error** (average Euclidean distance between predicted and true corners, in pixels) and a stricter success metric such as the fraction of images where all four corners fall within a small threshold of the ground truth. Which approach is more accurate? Which is more robust to unusual viewpoints? Which was easier to train? Support your verdict with numbers and failure-case visualizations.



## 5.1. Pipeline the process

As with the enhancement network, you should create a pipeline to automate the inference process for corner detection. This function will take a raw document photo as input and perform the following steps:

1. **Preprocess the image:** Resize and normalize the input photo as required by your model.
2. **Predict the four corners:** Pass the preprocessed image through your better trained model to obtain the predicted corner coordinates.
3. **Map coordinates:** Scale the predicted coordinates back to the original image resolution.
4. **Visualize the corners:** Overlay the predicted corners on the original raw photo.

 **Hint:** Think about *why* the two approaches might behave differently before running the experiments, and write your prediction down. Direct regression forces fully connected layers to map global features to precise coordinates; heatmaps keep the problem spatial and local. Was your prediction right?

---

## 6. Regularize Your Models Using Dropout

Now, update both of your models — the enhancement network and your corner detectors — by inserting Dropout layers and train them again to observe the difference in performance. For the direct-regression corner detector, the fully connected layers are the classic place for Dropout; for the encoder-decoder models, experiment with where in the architecture it helps. *In particular, does the gap between synthetic validation scores and real-photo test scores shrink?* Report the impact on both models.



## 7. Bonus: The End-to-End Document Scanner

You now have two trained networks and two inference pipelines that have never met, you will compose them into a single automatic scanner. Take the corner pipeline from Section 5 and compute the homography from its four predicted corners. In PyTorch, the kornia library provides differentiable `get_perspective_transform` and `warp_perspective` functions (alternatively, `torch.nn.functional.grid_sample` with a manually constructed grid); in plain OpenCV, `cv2.getPerspectiveTransform` and `cv2.warpPerspective` do the job. Feed the rectified crop into your trained enhancement network — you now have a complete document scanner that requires no human input. Evaluate the full chain on the real test photos and report the OCR metric and qualitative results twice: once rectifying with your annotated corners, and once with predicted corners. The difference tells you exactly how much corner errors cost the enhancement stage.

✖ **Option:** Since kornia's warp is differentiable, the ambitious among you can chain corner detector → warp → enhancement network and fine-tune the whole system end-to-end with the enhancement loss. Does the corner detector improve when it is trained for what the pipeline actually needs? Does the gap you measured above shrink?

💡 **Hint:** Watch out for a subtle failure mode: if the predicted corners are in the wrong order, the homography will flip or rotate the page. Your consistent corner ordering from Section 1.2 pays off here.

## Submission Criteria

To ensure a successful evaluation of your document scanning project, the following criteria must be met in your submission:

- **Code Implementation & Explanation**
  - Provide a well-documented, modular, and executable codebase.
  - Demonstrate a strong grasp of all concepts (e.g., synthetic data generation, model architecture, loss functions, post-processing).
  - Be prepared to explain and modify any part of the code if asked (e.g., adjusting hyperparameters, changing the model architecture, adding a new degradation).
- **Visualization of Results**
  - Visualize intermediate and final outputs (e.g., degraded input, enhanced output, clean target on synthetic data, reference scan on real photos, predicted corners).
  - Include comparisons between different methods (loss functions, and regression vs. heatmap) with qualitative analysis.
- **Pipeline for Unseen Data**
  - Provide two inference pipelines: one accepting an unseen rectified document image (enhancement), one accepting an unseen raw photo (corner detection). A single fully automatic photo-to-scan pipeline is the bonus.
  - Ensure the pipeline is robust to variations (e.g., lighting, shadows, distance, different backgrounds).
- **Performance Metrics & Analysis**
  - Report PSNR and SSIM on the synthetic training, validation, and test splits, alongside a no-model baseline; on real photos, report OCR-based readability improvement and a qualitative comparison against the commercial scanning app.
  - Discuss limitations and potential improvements (e.g., curled or folded pages, extreme shadows, the synthetic-to-real gap).

 **Note:** Submissions that fail to meet these criteria may be returned for revisions. Focus on clarity, functionality, and demonstrating deep understanding.



## References & Resources

- <https://docs.roboflow.com/annotate/annotation-tools/keypoint-annotation>
- <https://kornia.readthedocs.io/en/latest/geometry.transform.html>
- <https://github.com/tesseract-ocr/tesseract>